

Practicing cheatsheet codes of Pandas_Library

What is Pandas Library?

Pandas is a powerful, open-source (means something is publicly accessible, and can be used, modified, or shared by anyone. The term is most often used to describe open source software) Python library widely used for **data manipulation** and **analysis**. It provides easy-to-use data structures like DataFrames (think of them as enhanced spreadsheets) and Series (like single columns from a spreadsheet), along with tools to efficiently work with this data.

Why is Pandas Used?

Pandas excels at handling tabular data (like spreadsheets or data from databases), making it a cornerstone in data science. Here's why it's so popular:

- **Data Cleaning** : Pandas helps you deal with missing values, remove duplicates, and format your data consistently.
- **Data Transformation** : You can easily reshape, merge, split, and filter your data to get it into the exact form you need.
- **Data Analysis** : Pandas provides tools for calculating statistics, grouping data, and performing other analyses.
- **Integration** : Pandas works seamlessly with other popular Python libraries like NumPy (for numerical operations) and Matplotlib (for visualization).

Who Created and Maintains Pandas?

Creator [<https://wesmckinney.com/pages/about.html>]: Wes McKinney originally developed Pandas at AQR Capital Management. **Maintainers** : It's now an active open-source project with a community of contributors. You can find its source code and contribute on GitHub.

Why Learn Pandas?

Pandas is essential for tasks like data preprocessing, exploratory data analysis (EDA), and feature engineering in machine learning. It integrates seamlessly with libraries like NumPy, Matplotlib, and Scikit-learn, forming the backbone of Python's data ecosystem.

Note that:

To explore the pandas library, what it is how it works, how to consume the thing in the library to make the data ready to use. For all this we have to first call or install the library of pandas.

```
In [1]: # Load the pandas Library
import pandas as pd
```

1. Creating Data Frames using Pandas Library.

- **Data Frames** : A 2-dimensional labeled data structure with columns of potentially different types.
 - **Rows** : Represented by index labels, has a unique value for each row in columns.
 - **Columns** : Represented by column labels, has a values of same characteristics of interests.
 - **Two-dimensional** : Anything that has arrangement in row and column can be thought of as a separate dimension.

```
In [2]: # We Create a dataframe using pandas Library
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
df
```

```
Out[2]:
```

	A	B	C
0	1	4	7
1	2	5	8
2	3	6	9

```
In [3]: # secondly we create a dataframe that has a specific value of columns and rows
df_1 = pd.DataFrame(
    data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]],
    index = ["row1", "row2", "row3"],
    columns = ["col1", "col2", "col3"]
)
df_1
```

```
Out[3]:
```

	col1	col2	col3
row1	1	2	3
row2	4	5	6
row3	7	8	9

```
In [4]: # Data frame of a bio data of students studying college, with a row index of the st
# age,height,weight,gender,year of study,main_course,city,Distirct,State,Pincode an
```

```
df_2 = pd.DataFrame({
    'Name': ['Adil', 'Waleed', 'Osama', 'Abdul', 'Arafat', 'Ali', 'Yasir', 'Hafeez',
            'Ammal', 'Ammar', 'Faizan', 'Fahad', 'Faisal', 'Fahim', 'Fahad'],
    'Age': [20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37],
    'Height_in_m': [160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172,
                    173, 174, 175, 176, 177],
    'Weight_kg': [50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67],
    'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F'],
    'Year_of_Study': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Main_Course': ['Maths', 'Computer', 'Zoology', 'Botany', 'Statistics', 'Chemistry', 'Physics',
                    'Maths', 'Computer', 'Zoology', 'Botany', 'Statistics', 'Chemistry', 'Physics',
                    'Maths', 'Computer', 'Zoology', 'Botany', 'Statistics', 'Chemistry', 'Physics'],
    'cities': ["Karachi", "Lahore", "Islamabad", "Rawalpindi", "Faisalabad", "Multan", "Peshawar",
               "Hyderabad", "Sargodha", "Bahawalpur", "Sukkur", "Larkana", "Sheikhupura", "Faisalabad"],
    'districts': ["Karachi", "Lahore", "Islamabad", "Rawalpindi", "Faisalabad", "Multan", "Peshawar",
                  "Hyderabad", "Sargodha", "Bahawalpur", "Sukkur", "Larkana", "Sheikhupura", "Faisalabad"],
    'states': ["Sindh", "Punjab", "Islamabad", "Punjab", "Punjab", "Punjab", "Khyber Pakhtunkhwa",
               "Sindh", "Punjab", "Punjab", "Sindh", "Sindh", "Punjab", "Punjab", "Punjab", "Khyber Pakhtunkhwa"],
    'Pincode': ["123456", "234567", "345678", "456789", "567890", "678901", "789012", "890123",
                 "901234", "012345", "123456", "234567", "345678", "456789", "567890", "678901", "789012", "890123", "901234", "012345"],
    'Minimum Marks': [50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67]
})
df_2
```

Out[4]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

2. Reshaping data using Pandas Library.

- **Reshaping** : This is a process of changing the shape of the data. It can be done in many ways, some them are discussed below:
 - **Pivoting** : This is a process of transforming data from long format to wide format.
 - **Merging** : This is a process of combining data from two or more dataframes.
 - **Stack** : This is a process of stacking data from multiple rows into a single row.
 - **Unstack** : This is a process of unstacking data from a single row into multiple rows

Melting_Dataframe

Melting a DataFrame in Pandas refers to transforming it from a **wide format** to a **long format**, making it more suitable for analysis and visualization. It reshapes the data by converting column headers into row values, typically using the `pd.melt()` function. This is useful for unpivoting data, making it easier to group, filter, or plot.

In [5]: *# Melting of the data frame using pandas library*

```
df_3 = pd.melt(  
    df_2,  
    id_vars = ['Name'],  
    value_vars = ['Age', 'Height_in_m', 'Weight_kg'],  
    var_name = 'Attributes', value_name = 'Values'  
)  
df_3
```

Out[5]:

	Name	Attributes	Values
0	Adil	Age	20
1	Waleed	Age	21
2	Osama	Age	22
3	Abdul	Age	23
4	Arafat	Age	24
5	Ali	Age	25
6	Yasir	Age	26
7	Hafeez	Age	27
8	Raza	Age	28
9	Asad	Age	29
10	Ahmed	Age	30
11	Ammar	Age	31
12	Ammad	Age	32
13	Ammal	Age	33
14	Ammar	Age	34
15	Faizan	Age	35
16	Fahad	Age	36
17	Faisal	Age	37
18	Fahim	Age	38
19	Fahad	Age	39
20	Adil	Height_in_m	160
21	Waleed	Height_in_m	161
22	Osama	Height_in_m	162
23	Abdul	Height_in_m	163
24	Arafat	Height_in_m	164
25	Ali	Height_in_m	165
26	Yasir	Height_in_m	166
27	Hafeez	Height_in_m	167
28	Raza	Height_in_m	168
29	Asad	Height_in_m	169

	Name	Attributes	Values
30	Ahmed	Height_in_m	170
31	Ammar	Height_in_m	171
32	Ammad	Height_in_m	172
33	Ammal	Height_in_m	173
34	Ammar	Height_in_m	174
35	Faizan	Height_in_m	175
36	Fahad	Height_in_m	176
37	Faisal	Height_in_m	177
38	Fahim	Height_in_m	178
39	Fahad	Height_in_m	179
40	Adil	Weight_kg	50
41	Waleed	Weight_kg	51
42	Osama	Weight_kg	52
43	Abdul	Weight_kg	53
44	Arafat	Weight_kg	54
45	Ali	Weight_kg	55
46	Yasir	Weight_kg	56
47	Hafeez	Weight_kg	57
48	Raza	Weight_kg	58
49	Asad	Weight_kg	59
50	Ahmed	Weight_kg	60
51	Ammar	Weight_kg	61
52	Ammad	Weight_kg	62
53	Ammal	Weight_kg	63
54	Ammar	Weight_kg	64
55	Faizan	Weight_kg	65
56	Fahad	Weight_kg	66
57	Faisal	Weight_kg	67
58	Fahim	Weight_kg	68
59	Fahad	Weight_kg	69

3. *Pivot Tables using Pandas Library.*

- `Pivot_tables` : A pivot table is a data summarization tool used to group, aggregate, and analyze large datasets efficiently. In Pandas, pivot tables allow you to restructure and summarize data based on different categories, similar to Excel pivot tables.

In [6]: `pip install seaborn`

Requirement already satisfied: seaborn in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (0.13.2)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from seaborn) (2.2.2)
Requirement already satisfied: pandas>=1.2 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from seaborn) (2.2.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: six>=1.5 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from seaborn) (2.2.2)
Requirement already satisfied: pandas>=1.2 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from seaborn) (2.2.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\programdata\miniconda3\envs\py_ed
a\lib\site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in c:\programdata\miniconda3\envs\py_e
da\lib\site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: six>=1.5 in c:\programdata\miniconda3\envs\py_ed
a\lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.
0)

```
In [7]: import seaborn as sns
import pandas as pd

# Load the Titanic dataset
titanic_df = sns.load_dataset("titanic")

# Creating a pivot table to analyze survival rates by class and gender
pivot_table = titanic_df.pivot_table(
    values="survived", # The column to aggregate (0 = Died, 1 = Survived)
    index="class",     # Rows (Index) -> Passenger Class (First, Second, Third)
    columns="sex",     # Columns -> Gender (Male, Female)
    aggfunc="mean"     # Aggregation function -> Mean (Survival Rate)
)

# Display the pivot table
pivot_table
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_3844\1099751396.py:8: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior

```
    pivot_table = titanic_df.pivot_table(
```

```
Out[7]:
```

	sex	female	male
class			
First		0.968085	0.368852
Second		0.921053	0.157407
Third		0.500000	0.135447

4. Concatinating of data using Pandas Library.

- Concatination : Concatenating data means combining two or more data structures into one. Pandas library provides a function `concat()` to concatenate data. It can be used to concatenate DataFrames, Series, or even other data structures.

```
In [8]: # Here we simply use the concat commant to join the two dataframes into a single da
# Creating first DataFrame
df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie']
})

# Creating second DataFrame
```

```
df2 = pd.DataFrame({
    'ID': [4, 5, 6],
    'Name': ['David', 'Eva', 'Frank']
})

# Concatenating DataFrames Vertically (Row-wise)
vertical_concat = pd.concat([df1, df2], axis=0, ignore_index=True)
vertical_concat
```

Out[8]:

	ID	Name
0	1	Alice
1	2	Bob
2	3	Charlie
3	4	David
4	5	Eva
5	6	Frank

In [9]:

```
# Concatenating DataFrames Horizontally (Column-wise)
horizontal_concat = pd.concat([df1, df2], axis=1)
horizontal_concat
```

Out[9]:

	ID	Name	ID	Name
0	1	Alice	4	David
1	2	Bob	5	Eva
2	3	Charlie	6	Frank

5. Sort values using Pandas Library.

- **Sort_values** : This function takes a DataFrame as input and returns a new DataFrame with the sorted values of each column.

In [10]:

```
# sort the data frame by the values of the column 'Age'
df_2.sort_values('Age')
```

Out[10]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpin
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [11]: `# sort the data frame by the values of the column 'Age' in descending order`
`df_2.sort_values('Age', ascending=False)`

Out[11]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
19	Fahad	39	179	69	F	10	Chemistry	Abbottab
18	Fahim	38	178	68	M	9	Bilogy	Marc
17	Faisal	37	177	67	F	8	Biology	Guj
16	Fahad	36	176	66	M	7	Physics	Jha
15	Faizan	35	175	65	F	6	Chemistry	Sheikhpu
14	Ammar	34	174	64	M	5	Statistics	Larka
13	Ammal	33	173	63	F	4	Botany	Sukl
12	Ammad	32	172	62	M	3	Zology	Bahawalp
11	Ammar	31	171	61	F	2	Computer	Sargoc
10	Ahmed	30	170	60	M	1	Maths	Hyderab
9	Asad	29	169	59	F	10	Chemistry	Gujranw
8	Raza	28	168	58	M	9	Bilogy	Siall
7	Hafeez	27	167	57	F	8	Biology	Que
6	Yasir	26	166	56	M	7	Physics	Peshav
5	Ali	25	165	55	F	6	Chemistry	Mult
4	Arafat	24	164	54	M	5	Statistics	Faisalab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
2	Osama	22	162	52	M	3	Zology	Islamab
1	Waleed	21	161	51	F	2	Computer	Lahc
0	Adil	20	160	50	M	1	Maths	Kara

6. Rename column name using Pandas Library.

- **Rename** : This function is used to rename the column name of a DataFrame. It takes two parameters: **old name** and **new name** .

```
In [12]: # Renaming the name of the colomns in the data frame.
df_2_rename=df_2.rename(
    columns = {'Name':'First_Name','Age':'Years_Old','Height_in_m':'Height in meter',
               'Year_of_Study':'Year of Study', 'Main_Course':'Main Course'})
df_2_rename
```

Out[12]:

	First_Name	Years_Old	Height in meters	Weight in kg	M/F	Year of Study	Main Course	cities	district
0	Adil	20	160	50	M	1	Maths	Karachi	Karachi
1	Waleed	21	161	51	F	2	Computer	Lahore	Lahore
2	Osama	22	162	52	M	3	Zology	Islamabad	Islamabad
3	Abdul	23	163	53	F	4	Botany	Rawalpindi	Rawalpindi
4	Arafat	24	164	54	M	5	Statistics	Faisalabad	Faisalabad
5	Ali	25	165	55	F	6	Chemistry	Multan	Multan
6	Yasir	26	166	56	M	7	Physics	Peshawar	Peshawar
7	Hafeez	27	167	57	F	8	Biology	Quetta	Quetta
8	Raza	28	168	58	M	9	Bilogy	Sialkot	Sialkot
9	Asad	29	169	59	F	10	Chemistry	Gujranwala	Gujranwala
10	Ahmed	30	170	60	M	1	Maths	Hyderabad	Hyderabad
11	Ammar	31	171	61	F	2	Computer	Sargodha	Sargodha
12	Ammad	32	172	62	M	3	Zology	Bahawalpur	Bahawalpur
13	Ammal	33	173	63	F	4	Botany	Sukkur	Sukkur
14	Ammar	34	174	64	M	5	Statistics	Larkana	Larkana
15	Faizan	35	175	65	F	6	Chemistry	Sheikhupura	Sheikhupura
16	Fahad	36	176	66	M	7	Physics	Jhang	Jhang
17	Faisal	37	177	67	F	8	Biology	Gujrat	Gujrat
18	Fahim	38	178	68	M	9	Bilogy	Mardan	Mardan
19	Fahad	39	179	69	F	10	Chemistry	Abbottabad	Abbottabad

7. Drop colomns using Pandas Library.

- `Drop_Names` : This function drops the specified columns from the dataframe.

```
In [13]: # Drop the colomn from the data frame
df_2.drop('Name', axis=1)
```

Out[13]:

	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cities	d
0	20	160	50	M	1	Maths	Karachi	
1	21	161	51	F	2	Computer	Lahore	
2	22	162	52	M	3	Zology	Islamabad	Isl
3	23	163	53	F	4	Botany	Rawalpindi	Rav
4	24	164	54	M	5	Statistics	Faisalabad	Fai
5	25	165	55	F	6	Chemistry	Multan	
6	26	166	56	M	7	Physics	Peshawar	Pe
7	27	167	57	F	8	Biology	Quetta	
8	28	168	58	M	9	Bilogy	Sialkot	
9	29	169	59	F	10	Chemistry	Gujranwala	Gujr
10	30	170	60	M	1	Maths	Hyderabad	Hyc
11	31	171	61	F	2	Computer	Sargodha	Sa
12	32	172	62	M	3	Zology	Bahawalpur	Baha
13	33	173	63	F	4	Botany	Sukkur	
14	34	174	64	M	5	Statistics	Larkana	
15	35	175	65	F	6	Chemistry	Sheikhupura	Sheik
16	36	176	66	M	7	Physics	Jhang	
17	37	177	67	F	8	Biology	Gujrat	
18	38	178	68	M	9	Bilogy	Mardan	
19	39	179	69	F	10	Chemistry	Abbottabad	Abbo

8. *Reset_Index using Pandas Library.*

- **Reset_Index** : A Pandas DataFrame method used to reset the index of a DataFrame, converting the current index into a regular column or completely discarding it. This is useful when dealing with filtered, sorted, or concatenated DataFrames where the index may be unordered or inconsistent.

```
In [14]: # Reset the index of the data frame.
df_2.reset_index()
```

Out[14]:

	index	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course
0	0	Adil	20	160	50	M	1	Maths
1	1	Waleed	21	161	51	F	2	Computer
2	2	Osama	22	162	52	M	3	Zology
3	3	Abdul	23	163	53	F	4	Botany
4	4	Arafat	24	164	54	M	5	Statistics
5	5	Ali	25	165	55	F	6	Chemistry
6	6	Yasir	26	166	56	M	7	Physics
7	7	Hafeez	27	167	57	F	8	Biology
8	8	Raza	28	168	58	M	9	Bilogy
9	9	Asad	29	169	59	F	10	Chemistry
10	10	Ahmed	30	170	60	M	1	Maths
11	11	Ammar	31	171	61	F	2	Computer
12	12	Ammad	32	172	62	M	3	Zology
13	13	Ammal	33	173	63	F	4	Botany
14	14	Ammar	34	174	64	M	5	Statistics
15	15	Faizan	35	175	65	F	6	Chemistry
16	16	Fahad	36	176	66	M	7	Physics
17	17	Faisal	37	177	67	F	8	Biology
18	18	Fahim	38	178	68	M	9	Bilogy
19	19	Fahad	39	179	69	F	10	Chemistry

9. Subsetting observations using Pandas Library.

- **Subsetting_Observations** : Subsetting rows involves selecting specific observations based on conditions or row positions. This can be done using **boolean indexing** (e.g., `df[df['Age'] > 30]` to filter rows where Age is greater than 30) or by using **iloc[]** for position-based selection (e.g., `df.iloc[0:3]` to select the first three rows). It helps in extracting relevant data for analysis.

In [15]: *# Drop the duplicate values from the data frame.*

```
df_2.drop_duplicates()
```

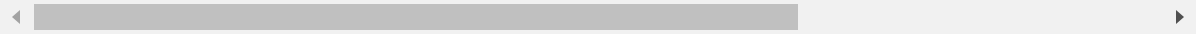
Out[15]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [16]: *# see the head and tail of the data frame.*
`df_2.head()`

Out[16]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cities
0	Adil	20	160	50	M	1	Maths	Karachi
1	Waleed	21	161	51	F	2	Computer	Lahore
2	Osama	22	162	52	M	3	Zology	Islamabad
3	Abdul	23	163	53	F	4	Botany	Rawalpindi
4	Arafat	24	164	54	M	5	Statistics	Faisalabad



In [17]: `df_2.tail()`

Out[17]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	citie
15	Faizan	35	175	65	F	6	Chemistry	Sheikhpur
16	Fahad	36	176	66	M	7	Physics	Jhan
17	Faisal	37	177	67	F	8	Biology	Gujra
18	Fahim	38	178	68	M	9	Bilogy	Marda
19	Fahad	39	179	69	F	10	Chemistry	Abbottaba



In [18]: *# See the sample from the data frame.*
`df_2.sample()`

Out[18]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cities
6	Yasir	26	166	56	M	7	Physics	Peshawar F



In [19]: `df_2.sample(n=5) # Randomly select 5 rows`

Out[19]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	city
9	Asad	29	169	59	F	10	Chemistry	Gujranwala
14	Ammar	34	174	64	M	5	Statistics	Larkana
11	Ammar	31	171	61	F	2	Computer	Sargodha
18	Fahim	38	178	68	M	9	Biology	Mardan
19	Fahad	39	179	69	F	10	Chemistry	Abbottabad

In [20]: `df_2.nlargest(5, 'Age') # Select the 5 largest values in the 'Age' column`

Out[20]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	city
19	Fahad	39	179	69	F	10	Chemistry	Abbottabad
18	Fahim	38	178	68	M	9	Biology	Mardan
17	Faisal	37	177	67	F	8	Biology	Gujranwala
16	Fahad	36	176	66	M	7	Physics	Jhang
15	Faizan	35	175	65	F	6	Chemistry	Sheikhupura

In [21]: `df_2.nsmallest(5, 'Age') # Select the 5 smallest values in the 'Age' column`

Out[21]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cities
0	Adil	20	160	50	M	1	Maths	Karachi
1	Waleed	21	161	51	F	2	Computer	Lahore
2	Osama	22	162	52	M	3	Zology	Islamabad
3	Abdul	23	163	53	F	4	Botany	Rawalpindi
4	Arafat	24	164	54	M	5	Statistics	Faisalabad

In [22]: `df_2[df_2['Age'] > 25] # Select rows where the 'Age' column is greater than 25`

Out[22]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

10. Subsetting Along Columns.

- **Subsetting columns** means selecting specific attributes or features from the DataFrame. You can use **column names** (e.g., `df[['Name', 'Age']]` to select only these columns) or **range selection** using `loc` (e.g., `df.loc[:, 'Name':'Age']`). This is useful when focusing on specific variables for visualization or analysis.

```
In [23]: # Select the single column from the data frame.  
df_2['Name'] # Select the 'Name' column or df_2.Name.
```

```
Out[23]: 0      Adil
          1     Waleed
          2      Osama
          3      Abdul
          4     Arafat
          5       Ali
          6      Yasir
          7     Hafeez
          8       Raza
          9       Asad
         10     Ahmed
         11     Ammar
         12     Ammad
         13     Ammal
         14     Ammar
         15     Faizan
         16     Fahad
         17     Faisal
         18     Fahim
         19     Fahad
          Name: Name, dtype: object
```

```
In [24]: # Select multiple columns at once.
          df_2[['Name', 'Age', 'Height_in_m']]
```

Out[24]:

	Name	Age	Height_in_m
0	Adil	20	160
1	Waleed	21	161
2	Osama	22	162
3	Abdul	23	163
4	Arafat	24	164
5	Ali	25	165
6	Yasir	26	166
7	Hafeez	27	167
8	Raza	28	168
9	Asad	29	169
10	Ahmed	30	170
11	Ammar	31	171
12	Ammad	32	172
13	Ammal	33	173
14	Ammar	34	174
15	Faizan	35	175
16	Fahad	36	176
17	Faisal	37	177
18	Fahim	38	178
19	Fahad	39	179

```
In [25]: # Filter the data frame by the values of the column.  
df_2[df_2['Age'] > 25]
```

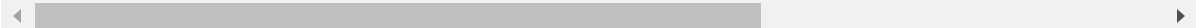
Out[25]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [26]: `df_2[df_2['Height_in_m'] > 170]`

Out[26]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhupu
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab



In [27]: `df_2[df_2['Weight_kg'] > 60]`

Out[27]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhupu
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab



In [28]: `df_2[df_2['Gender'] == 'M']`

Out[28]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	city
0	Adil	20	160	50	M	1	Maths	Karac
2	Osama	22	162	52	M	3	Zology	Islamaba
4	Arafat	24	164	54	M	5	Statistics	Faisalaba
6	Yasir	26	166	56	M	7	Physics	Peshaw
8	Raza	28	168	58	M	9	Bilogy	Sialk
10	Ahmed	30	170	60	M	1	Maths	Hyderaba
12	Ammad	32	172	62	M	3	Zology	Bahawalp
14	Ammar	34	174	64	M	5	Statistics	Larkar
16	Fahad	36	176	66	M	7	Physics	Jhar
18	Fahim	38	178	68	M	9	Bilogy	Marda

Quary Function using Python Library.

Query_Function : The **query()** function in Pandas is used to filter rows in a DataFrame using a string-based condition, making the syntax cleaner and more readable compared to boolean indexing. For example, `df.query("Age > 30 and City == 'New York'")` selects rows where **Age is greater than 30 and City is 'New York'**. This method is particularly useful when working with large datasets and complex filtering conditions.

```
In [29]: # Query Function using python Library.  
df_2.query('Age > 25')
```

Out[29]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

11. Subsetting Along Rows and Columns.

- Combining row and column subsetting allows extracting specific data points from a DataFrame. You can use `loc[]` for label-based filtering (e.g., `df.loc[df['Age'] > 30, ['Name', 'City']]` to select only 'Name' and 'City' for rows where Age > 30). Alternatively, `iloc[]` allows position-based selection (e.g., `df.iloc[1:4, 0:2]` to select specific row and column positions). This technique is useful for targeted data extraction.

```
In [30]: # Here we access the rows using the loc function.  
df_2.loc[1] # Select the first row
```

```
Out[30]: Name          Waleed
Age          21
Height_in_m  161
Weight_kg    51
Gender       F
Year_of_Study 2
Main_Course   Computer
cities        Lahore
districts     Lahore
states        Punjab
Pincode       234567
Minimum Marks 51
Name: 1, dtype: object
```

```
In [31]: # Here we access the columns using the iloc function.
df_2.iloc[0:5] # Select the first six columns
```

```
Out[31]:
```

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cities
0	Adil	20	160	50	M	1	Maths	Karachi
1	Waleed	21	161	51	F	2	Computer	Lahore
2	Osama	22	162	52	M	3	Zology	Islamabad
3	Abdul	23	163	53	F	4	Botany	Rawalpindi
4	Arafat	24	164	54	M	5	Statistics	Faisalabad



```
In [32]: # Here we access the columns using the iloc function.
df_2.iloc[0:5, 0:3] # Select the first six column and first three rows.
```

```
Out[32]:
```

	Name	Age	Height_in_m
0	Adil	20	160
1	Waleed	21	161
2	Osama	22	162
3	Abdul	23	163
4	Arafat	24	164

```
In [33]: # Select the single and entry of row.
df_2.at[7, 'Name']
```

```
Out[33]: 'Hafeez'
```

```
In [34]: # Select the single and entry of column.
df_2.iat[5, 7]
```

```
Out[34]: 'Multan'
```

12. Summarizing the dataset using Python Library.

- **Summarizing the dataset** : We can use the pandas library to summarize the dataset. We can use the `describe()` function to get a summary of the dataset, including the count, mean, standard deviation, minimum, maximum and quartiles.

```
In [35]: # Count the values in column, means the number of each value in the column that how  
df_2['Age'].value_counts()
```

```
Out[35]: Age  
20      1  
21      1  
22      1  
23      1  
24      1  
25      1  
26      1  
27      1  
28      1  
29      1  
30      1  
31      1  
32      1  
33      1  
34      1  
35      1  
36      1  
37      1  
38      1  
39      1  
Name: count, dtype: int64
```

```
In [36]: # See the shape of the data frame, how many rows and columns are in the data frame.  
df_2.shape
```

```
Out[36]: (20, 12)
```

```
In [37]: # See the unique values in the column of data frame.  
df_2['districts'].unique()
```

```
Out[37]: array(['Karachi', 'Lahore', 'Islamabad', 'Rawalpindi', 'Faisalabad',  
                'Multan', 'Peshawar', 'Quetta', 'Sialkot', 'Gujranwala',  
                'Hyderabad', 'Sargodha', 'Bahawalpur', 'Sukkur', 'Larkana',  
                'Sheikhupura', 'Jhang', 'Gujrat', 'Mardan', 'Abbottabad'],  
              dtype=object)
```

```
In [38]: # Summary of the columns in the data frame.  
df_2.describe()
```

Out[38]:

	Age	Height_in_m	Weight_kg	Year_of_Study	Minimum Marks
count	20.00000	20.00000	20.00000	20.000000	20.00000
mean	29.50000	169.50000	59.50000	5.500000	59.50000
std	5.91608	5.91608	5.91608	2.946898	5.91608
min	20.00000	160.00000	50.00000	1.000000	50.00000
25%	24.75000	164.75000	54.75000	3.000000	54.75000
50%	29.50000	169.50000	59.50000	5.500000	59.50000
75%	34.25000	174.25000	64.25000	8.000000	64.25000
max	39.00000	179.00000	69.00000	10.000000	69.00000

Note:

Pandas provides a large set of summary functions that operate on different kinds of pandas objects (DataFrame columns, Series, GroupBy, Expanding and Rolling (see below)) and produce single values for each of the groups. When applied to a DataFrame, the result is returned as a pandas Series for each column.

1. **sum()** function : This function returns the sum of all elements in the given iterable.
Sum of all elements in the list is $10 + 20 + 30 + 40 + 50 = 150$.

```
In [39]: df_2['Weight_kg'].sum()
```

```
Out[39]: np.int64(1190)
```

2. **count()** function : This function returns the number of elements in the list. It is a built-in function in Python.

```
In [40]: df_2['Weight_kg'].count()
```

```
Out[40]: np.int64(20)
```

3. **median()** function : This function returns the median of the given data. The median is the middle value in a sorted list. If the list has an even number of values, the median is the average of the two middle values.

```
In [41]: df_2['Weight_kg'].median()
```

```
Out[41]: np.float64(59.5)
```

4. `quantile()` function : This function is used to calculate the specified quantile of a dataset. It returns the specified quantiles. Quantiles are the values at which the data can be divided into equal parts. For example, the 25 th percentile is the value below which 25% of the data falls. The 50th percentile is the median. The 75th percentile is the value below which 75% of the data falls.

```
In [42]: df_2['Weight_kg'].quantile(0.25)
```

```
Out[42]: np.float64(54.75)
```

```
In [43]: df_2['Weight_kg'].quantile(0.50)
```

```
Out[43]: np.float64(59.5)
```

```
In [44]: df_2['Weight_kg'].quantile(0.75)
```

```
Out[44]: np.float64(64.25)
```

5. `IQR()` function : This function is used to calculate the interquartile range (IQR) of a dataset. The IQR is the difference between the 75th percentile (Q3) and the 25 th percentile (Q1). It is a measure of the spread or dispersion of the data. The IQR is calculated as follows: $IQR = Q3 - Q1$. The IQR is a useful measure of the spread of the data, as it is less affected by outliers than the mean or standard deviation.

```
In [45]: df_2['Weight_kg'].quantile(0.75) - df_2['Weight_kg'].quantile(0.25)
```

```
Out[45]: np.float64(9.5)
```

```
In [46]: pip install numpy
```

Requirement already satisfied: numpy in c:\programdata\miniconda3\envs\py_eda\lib\site-packages (2.2.2)Note: you may need to restart the kernel to use updated packages.

```
In [47]: import numpy as np
```

6. `apply()` function: This function is used to apply a function to each element of an array.

```
In [48]: df_2['Weight_kg'].apply(np.sqrt)
```

```
Out[48]: 0      7.071068
         1      7.141428
         2      7.211103
         3      7.280110
         4      7.348469
         5      7.416198
         6      7.483315
         7      7.549834
         8      7.615773
         9      7.681146
        10      7.745967
        11      7.810250
        12      7.874008
        13      7.937254
        14      8.000000
        15      8.062258
        16      8.124038
        17      8.185353
        18      8.246211
        19      8.306624
        Name: Weight_kg, dtype: float64
```

6. `min()` function : This function returns the smallest item in an iterable or the smallest of two or more arguments.

```
In [49]: df_2['Weight_kg'].min()
```

```
Out[49]: np.int64(50)
```

7. `max()` function : This function returns the largest item in an iterable or the largest of two or more arguments.

```
In [50]: df_2['Weight_kg'].max()
```

```
Out[50]: np.int64(69)
```

8. `mean()` function : This function calculates the mean of a given array. It takes an array as an argument and returns the mean of the array. Mean is the average of all the numbers in a dataset. It is calculated by adding up all the numbers and then dividing by the count of numbers. For example, if we have the numbers 1, 2, 3, 4, and 5, the mean would be $(1 + 2 + 3 + 4 + 5) / 5 = 3$.

```
In [51]: df_2['Weight_kg'].mean()
```

```
Out[51]: np.float64(59.5)
```

9. `Var()` function : This function returns the variance of the data. It is used to calculate the variance of the data. variance is generally used to measure the spread of the data. It

is calculated as the average of the squared differences from the mean. The formula for variance is: $\text{variance} = \sum (x_i - \mu)^2 / (n - 1)$ where x_i is the individual data point, μ is the mean, and n is the number of data points. The variance is a measure of the spread of the data and is used in many statistical tests. The variance is always non-negative and is zero if and only if all the data points are equal.

```
In [52]: df_2['Weight_kg'].var()
```

```
Out[52]: np.float64(35.0)
```

10. **Std() function** : This function returns the standard deviation of the data. It is used to calculate the standard deviation of the data. Standard deviation is a measure of the amount of variation or dispersion of a set of values. A low standard deviation indicates that the values tend to be close to the mean (also called the expected value) of the data set, while a high standard deviation indicates that the values are spread out over a wider range. The formula for standard deviation is: $\text{std} = \sqrt{\sum (x_i - \text{mean})^2 / (n - 1)}$ where x_i is each value in the data set, mean is the mean of the data set, n is the number of values in the data set, and $\sqrt{}$ is the square root function.

```
In [53]: df_2['Weight_kg'].std()
```

```
Out[53]: np.float64(5.916079783099616)
```

13. Handling Missing Values using the Pandas Library

Handling Missing Values: Missing values are a common problem in data analysis. There are several ways to handle missing values, including :

- **Problem** : A dataset contains missing values, and you want to handle them using the **fillna()** function. However, you are unsure about the best approach to replace the missing values.
- **Solution** : You can use the **fillna()** function to replace missing values with a specific value, such as the mean, median, or mode of the column. For example, to replace missing values in a column named 'Age' with the mean of the column. **dropna()** function can be used to drop rows with missing values.

1. **dropna() function** : This function is used to remove rows with missing values. It can be used with the **axis** parameter to specify whether to remove rows or columns with missing values. By default, it removes rows with missing values.

```
In [54]: df_2.dropna()
```

Out[54]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

2. `fillna()` function : This function is used to replace missing values in a DataFrame with a specified value. It can be used to replace missing values with a specific value, or with a value calculated based on other columns. For example, if we have a DataFrame with a column 'Age' and we want to replace missing values with the average age of the non-missing values.

In [55]: `df_2.fillna(0)`

Out[55]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

14. Making New Colomns using Pandas Library.

- Making new columns using the pandas library is a common task in data manipulation. You can create a new column by assigning a value to a new column name and creating new colomn using the already present colomns in a DataFrame.

```
In [56]: # Assign() function is used to add new columns to the dataframe.  
df_2['Weight_g'] = df_2['Weight_kg'] * 1000  
df_2
```

Out[56]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpin
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [57]:

```
# qcut() function is used to create bins of equal size.
df_2['Weight_Category'] = pd.qcut(df_2['Weight_kg'], q=3, labels=['Light', 'Medium', 'Heavy'])
df_2
```

Out[57]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

Note:

Pandas provides a large set of vector functions that operate on all columns of a DataFrame or a single selected column (a pandas Series). These functions produce vectors of values for each of the columns, or a single Series for the individual Series.

```
In [58]: # max() function is used to find the maximum value in the column.  
df_2.max(axis=0)
```

```
Out[58]: Name          Yasir
Age             39
Height_in_m     179
Weight_kg       69
Gender          M
Year_of_Study   10
Main_Course     Zology
cities          Sukkur
districts       Sukkur
states          Sindh
Pincode         901234
Minimum Marks   69
Weight_g        69000
Weight_Category Heavy
dtype: object
```

```
In [59]: # min() function is used to find the minimum value in the column.
df_2.min(axis=0)
```

```
Out[59]: Name          Abdul
Age             20
Height_in_m     160
Weight_kg       50
Gender          F
Year_of_Study   1
Main_Course     Bilogy
cities          Abbottabad
districts       Abbottabad
states          Balochistan
Pincode         012345
Minimum Marks   50
Weight_g        50000
Weight_Category Light
dtype: object
```

```
In [60]: # abs() function is used to find the absolute value of the column.
df_2['Weight_kg'].abs()
```

```
Out[60]: 0      50
         1      51
         2      52
         3      53
         4      54
         5      55
         6      56
         7      57
         8      58
         9      59
        10      60
        11      61
        12      62
        13      63
        14      64
        15      65
        16      66
        17      67
        18      68
        19      69
        Name: Weight_kg, dtype: int64
```

```
In [61]: # clip() function is used to clip the values of the column.
        df_2['Weight_kg'].clip(lower=55, upper=65)
```

```
Out[61]: 0      55
         1      55
         2      55
         3      55
         4      55
         5      55
         6      56
         7      57
         8      58
         9      59
        10      60
        11      61
        12      62
        13      63
        14      64
        15      65
        16      65
        17      65
        18      65
        19      65
        Name: Weight_kg, dtype: int64
```

15. Combine Datasets using Python Library.

- Combine the data frame using the merge function. This will combine the two data frames based on the common column. In this case, we are merging the data frames df1 and df2 dataset.

```
In [62]: df1 = pd.DataFrame({
        'ID': [1, 2, 3],
        'Name': ['Alice', 'Bob', 'Charlie']}
    )

    df2 = pd.DataFrame({
        'ID': [1, 2, 3],
        'Name': ['David', 'Eva', 'Frank']}
    )
```

```
In [63]: # merge the two data frames using the merge function, along left axis.
pd.merge(df1, df2, on='ID', how='left')
```

```
Out[63]:
```

	ID	Name_x	Name_y
0	1	Alice	David
1	2	Bob	Eva
2	3	Charlie	Frank

```
In [64]: # merge the two data frames using the merge function, along right axis.
pd.merge(df1, df2, on='ID', how='right')
```

```
Out[64]:
```

	ID	Name_x	Name_y
0	1	Alice	David
1	2	Bob	Eva
2	3	Charlie	Frank

```
In [65]: # merge the two data frames using the merge function, along inner axis.
pd.merge(df1, df2, on='ID', how='inner')
```

```
Out[65]:
```

	ID	Name_x	Name_y
0	1	Alice	David
1	2	Bob	Eva
2	3	Charlie	Frank

```
In [66]: # merge the two data frames using the merge function, along outer axis.
pd.merge(df1, df2, on='ID', how='outer')
```

```
Out[66]:
```

	ID	Name_x	Name_y
0	1	Alice	David
1	2	Bob	Eva
2	3	Charlie	Frank

In []:

16. Group Data using Pandas Library.

- `Groupby()` function : This function is used to group the data by one or more columns. It returns a `DataFrameGroupBy` object which can be used to perform various operations like aggregation, filtering, etc.

```
In [67]: # groupby() function for columns.  
df_2.groupby('Main_Course')['Weight_kg'].sum()
```

```
Out[67]: Main_Course  
Bilogy      126  
Biology      124  
Botany       116  
Chemistry    248  
Computer     112  
Maths        110  
Physics      122  
Statistics    118  
Zology       114  
Name: Weight_kg, dtype: int64
```

```
In [68]: # size function for group data.  
df_2.groupby('Main_Course').size()
```

```
Out[68]: Main_Course  
Bilogy      2  
Biology      2  
Botany       2  
Chemistry    4  
Computer     2  
Maths        2  
Physics      2  
Statistics    2  
Zology       2  
dtype: int64
```

```
In [69]: # aggregate function for group data.  
df_2.groupby('Main_Course').agg({'Weight_kg': 'sum', 'Age': 'mean'})
```

Out[69]:

	Weight_kg	Age
Main_Course		
Bilogy	126	33.0
Biology	124	32.0
Botany	116	28.0
Chemistry	248	32.0
Computer	112	26.0
Maths	110	25.0
Physics	122	31.0
Statistics	118	29.0
Zology	114	27.0

Note

- The examples below can also be applied to groups. In this case, the function is applied on a per-group basis, and the returned vectors are of the length of the original DataFrame.

```
In [70]: # use of shift function in group data.  
df_2['Previous_Weight'] = df_2.groupby('Main_Course')['Weight_kg'].shift(1)  
df_2
```

Out[70]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [71]:

```
df_2['Previous_Weight'] = df_2.groupby('Main_Course')['Weight_kg'].shift(-1)
df_2
```

Out[71]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [72]:

```
# use of rank function in group data.
df_2['Rank'] = df_2.groupby('Main_Course')['Weight_kg'].rank(method='min')
df_2
```

Out[72]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpin
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [73]:

```
df_2['Rank'] = df_2.groupby('Main_Course')['Weight_kg'].rank(pct=True)
df_2
```

Out[73]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [74]:

```
df_2['Rank'] = df_2.groupby('Main_Course')['Weight_kg'].rank(method='first')
df_2
```

Out[74]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpin
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [75]:

```
# cumsum() function is used to find the cumulative sum of the column.
df_2['Cumulative_Sum'] = df_2['Weight_kg'].cumsum()
df_2
```

Out[75]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [76]:

```
df_2['Cumulative_Sum'] = df_2['Weight_kg'].cummax()
df_2
```

Out[76]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [77]:

```
df_2['Cumulative_Sum'] = df_2['Weight_kg'].cummin()  
df_2
```

Out[77]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

In [78]: `df_2['Cumulative_Sum'] = df_2['Weight_kg'].cumprod()
df_2`

Out[78]:

	Name	Age	Height_in_m	Weight_kg	Gender	Year_of_Study	Main_Course	cit
0	Adil	20	160	50	M	1	Maths	Kara
1	Waleed	21	161	51	F	2	Computer	Lah
2	Osama	22	162	52	M	3	Zology	Islamab
3	Abdul	23	163	53	F	4	Botany	Rawalpi
4	Arafat	24	164	54	M	5	Statistics	Faisalab
5	Ali	25	165	55	F	6	Chemistry	Mult
6	Yasir	26	166	56	M	7	Physics	Peshav
7	Hafeez	27	167	57	F	8	Biology	Que
8	Raza	28	168	58	M	9	Bilogy	Siall
9	Asad	29	169	59	F	10	Chemistry	Gujranw
10	Ahmed	30	170	60	M	1	Maths	Hyderab
11	Ammar	31	171	61	F	2	Computer	Sargoc
12	Ammad	32	172	62	M	3	Zology	Bahawalp
13	Ammal	33	173	63	F	4	Botany	Sukl
14	Ammar	34	174	64	M	5	Statistics	Larka
15	Faizan	35	175	65	F	6	Chemistry	Sheikhup
16	Fahad	36	176	66	M	7	Physics	Jha
17	Faisal	37	177	67	F	8	Biology	Guj
18	Fahim	38	178	68	M	9	Bilogy	Marc
19	Fahad	39	179	69	F	10	Chemistry	Abbottab

17. Plotting using Python Library.

- **Plotting Data** : Plotting data is a crucial step in data analysis. It helps in visualizing the data and understanding the relationships between different variables. Python has several libraries that can be used for plotting data, including `matplotlib` and `seaborn` .

In [79]: *# Creating Histogram using pandas Library.*

```
import matplotlib.pyplot as plt
```

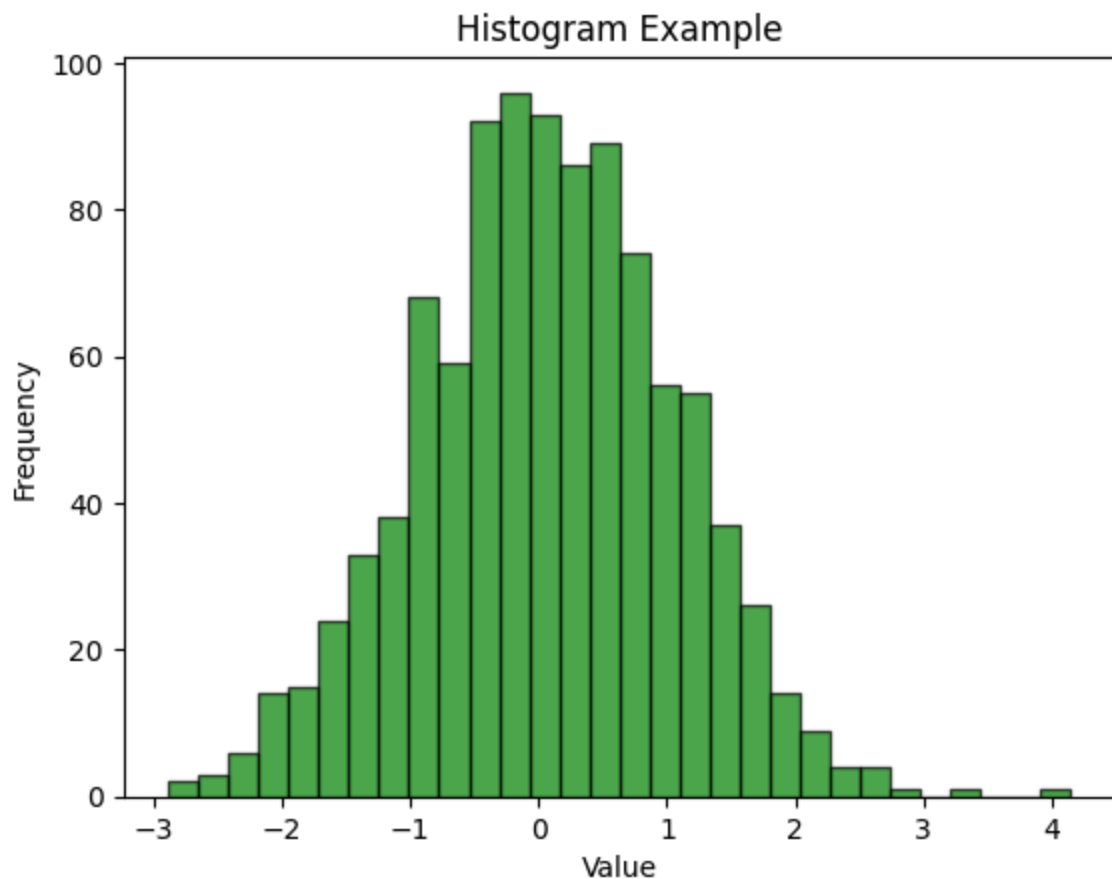
```
import numpy as np

# Sample data
data = np.random.randn(1000) # 1000 random values from a normal distribution

# Create histogram
plt.hist(data, bins=30, color='green', edgecolor='black', alpha=0.7)

# Labels and title
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram Example')

# Show plot
plt.show()
```



```
In [80]: # Creating Bar plot using pandas Library.
import matplotlib.pyplot as plt

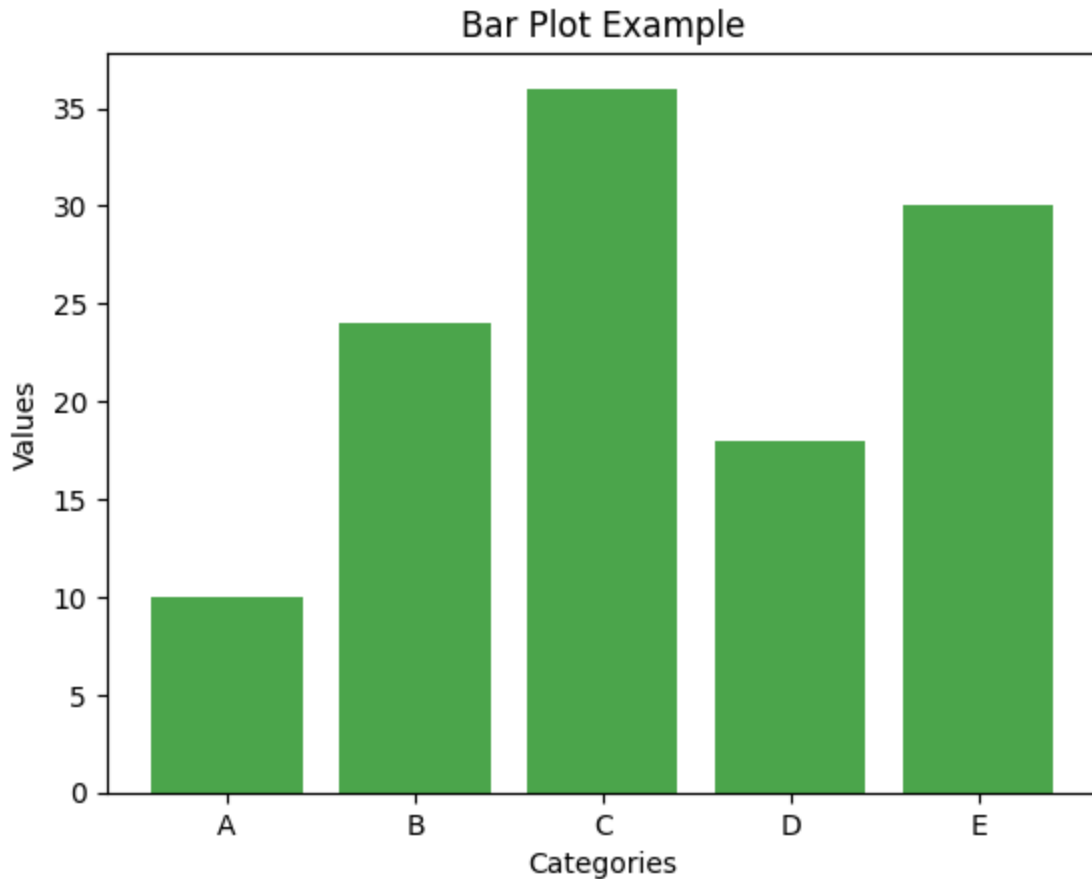
# Sample data
categories = ['A', 'B', 'C', 'D', 'E']
values = [10, 24, 36, 18, 30]

# Create bar plot
plt.bar(categories, values, color='green', alpha=0.7)

# Labels and title
plt.xlabel('Categories')
```

```
plt.ylabel('Values')
plt.title('Bar Plot Example')

# Show plot
plt.show()
```



```
In [81]: # Creating scatter plot using pandas library.
# Generate sample data
x = np.random.rand(100) * 10 # 100 random x values between 0 and 10
y = np.random.rand(100) * 10 # 100 random y values between 0 and 10

# Create scatter plot
plt.scatter(x, y, color='green', alpha=0.7, edgecolors='black')

# Labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Scatter Plot Example')

# Show plot
plt.show()
```

Scatter Plot Example

